

Neurocomment Bot

Архитектура мультиаккаунтного
Telegram-бота для AI-комментирования

Репозиторий: github.com/ImpulsRuZone/Marketix

Ветка: [cursor/full-architecture-bot-7336](#)

1. Общая идея

Neurocomment Bot — мультиаккаунтный Telegram-комментатор. Несколько Telegram-аккаунтов работают параллельно, каждый слушает целевые каналы. При появлении нового поста бот решает — комментировать или нет, генерирует текст через GPT-4o-mini и публикует комментарий в linked-группу канала. Все данные хранятся в Supabase/PostgreSQL.

Стек технологий

Компонент	Технология
Telegram API	Telethon (StringSession)
AI	OpenAI GPT-4o-mini
БД	PostgreSQL / Supabase (asyncpg)
Параллелизм	asyncio
Деплой	Docker, systemd

2. Высокоуровневая схема

```
main.py
├─ asyncio.gather(AccountWorker × N)
│   └─ Telethon Client (StringSession из БД)
│   └─ post_listener → NewMessage handler
│   └─ join_manager → вступление в каналы (фон)
│   └─ при новом посте:
│       └─ comment_scheduler (лимит + % + сон)
│       └─ random delay 60-300 сек
│       └─ generator → GPT-4o-mini
│       └─ comment_sender → linked-группа
```

3. Точка входа — main.py

При запуске бот выполняет следующие шаги:

- Читает .env (DATABASE_URL, API_ID, API_HASH, OPENAI_API_KEY)
- Инициализирует пул соединений с БД (asyncpg)
- Синхронизирует account_settings для всех аккаунтов
- Загружает активные аккаунты из таблицы accounts
- Создает по одному AccountWorker на каждый аккаунт
- Запускает все воркеры через asyncio.gather() — полностью параллельно

Режим вступления в каналы

Флаг / переменная	Поведение
-------------------	-----------

--join	Вступать в новые каналы при старте
--no-join	Только слушать уже известные каналы
JOIN_ON_STARTUP=true	Для systemd: вступать при рестарте
JOIN_ON_STARTUP=false	Для systemd: не вступать (по умолчанию)

4. Воркер аккаунта — AccountWorker

Центральный компонент. Один экземпляр = один Telegram-аккаунт. При сбое воркер автоматически перезапускается через 30 секунд.

Жизненный цикл при старте

- get_settings() — загрузка настроек из БД
- client.connect() — подключение Telethon
- is_user_authorized() — проверка сессии
- _wait_if_sleeping() — ожидание окончания окна сна
- get_joinable_target_chats() — список целевых каналов
- resolve_monitored_ids() — ID каналов для прослушивания
- register_post_handler() — регистрация обработчика постов
- join_all_chats() в фоне (если join_on_startup=true)
- run_until_disconnected() — основной цикл

Важно: прослушивание постов начинается сразу, вступление в новые каналы идёт в фоне — бот не ждёт окончания join перед комментированием.

Обработка нового поста

- Обновляет настройки из БД (можно менять лимиты без перезапуска)
- Проверяет is_active — аккаунт включён?
- Проверяет окно сна (sleep_start_time / sleep_end_time + timezone)
- Планировщик — комментировать или пропустить?
- Случайная задержка 60–300 сек (имитация человека)
- Сохраняет пост в БД (target_chats + posts)
- GPT генерирует комментарий
- Отправляет комментарий в linked-группу
- При ошибке доступа — исключает канал из мониторинга для этого аккаунта

5. Модули Telegram

client_factory.py

Создаёт TelegramClient из записи в БД:

- Сессия — StringSession (хранится в БД, не на диске)
- API_ID / API_HASH — общие для всех аккаунтов из .env

- Опционально — proxy (SOCKS5/HTTP) per account

post_listener.py

Регистрирует обработчик events.NewMessage с фильтрами:

- Только каналы (event.is_channel)
- Только из monitored_ids
- Только посты (event.message.post)
- Только текст ≥ 50 символов (MIN_POST_LENGTH)

join_manager.py

Вступление в каналы:

- Берёт список target_chats из БД
- Пропускает уже joined и excluded
- Вступает в канал + его linked-группу (для комментариев)
- Записывает статус в account_chats (pending → joined / failed / excluded)
- Случайная задержка между вступлениями (120–600 сек по умолчанию)

6. Пайплайн комментариев

comment_scheduler.py — решение «комментировать?»

- Если сегодня уже отправлено $\geq \text{max_comments_per_day}$ → пропуск
- Случайный бросок: комментировать с вероятностью $\text{daily_comment_percent} / 100$ (по умолчанию 30%)

generator.py — генерация текста

```
Модель: GPT-4o-mini, temperature=0.9
system_prompt = account.gpt_prompt or DEFAULT_GPT_PROMPT
user_message = post_text[:1000]
Результат обрезается до MAX_COMMENT_LENGTH (200 символов)
```

У каждого аккаунта может быть свой GPT prompt в таблице accounts.gpt_prompt.

comment_sender.py — отправка

Telegram не позволяет комментировать напрямую в канал — комментарий идёт в linked discussion group. Алгоритм отправки:

- GetFullChannelRequest → получить linked_chat_id
- Сохранить запись в comments (status=generated)
- JoinChannelRequest в linked-группу
- Попытка 1: event.respond()
- Попытка 2: send_message(comment_to=msg_id)
- Fallback: GetDiscussionMessageRequest → reply в linked-группе
- Fallback: скан linked-группы (100 msg) → reply на forwarded post

- При успехе: status=sent; при ошибке: status=failed

Принцип: отправка в Telegram независима от БД — запись в БД best-effort, не блокирует отpravku.

7. База данных

Таблицы

Таблица	Назначение
accounts	Telegram-аккаунты (session, proxy, gpt_prompt)
account_settings	Лимиты, окно сна, timezone, is_active
target_chats	Целевые каналы (@username или URL)
account_chats	Статус вступления аккаунта в канал (M:N)
posts	Найденные посты в каналах
comments	Сгенерированные и отправленные комментарии
logs	Системные события (INFO / WARNING / ERROR)

Представление comments_readable

Удобная таблица для Supabase UI — без UUID, время в Москве, читаемые имена аккаунтов и каналов.

Колонка	Описание
sent_at_moscow	Время отправки (МСК)
account_name	Имя аккаунта
chat_title	Название канала
post_text	Текст поста
generated_comment	Сгенерированный GPT комментарий
status	sent / failed / generated
sent_comment	Фактически отправленный текст

8. Настройки per account

В account_settings (можно менять в Supabase без перезапуска):

Параметр	По умолчанию	Назначение
daily_comment_percent	30	% постов, на которые реагируем
max_comments_per_day	20	Жёсткий лимит в день
sleep_start_time	null	Начало «сна» (например 23:00)
sleep_end_time	null	Конец «сна» (например 08:00)

timezone	UTC	Часовой пояс для сна
join_delay_min/max	120 / 600	Задержка между вступлениями (сек)
is_active	true	Вкл/выкл аккаунт

9. Обработка ошибок

Автоисключение каналов

При ошибках доступа канал исключается из мониторинга для аккаунта:

- private and you lack permission
- chat_write_forbidden
- you were banned

Статус excluded в account_chats — бот больше не тратит ресурсы на этот канал.

Перезапуск воркера

При необработанном исключении воркер перезапускается через 30 сек (цикл while True в run()).

10. Добавление аккаунта

```
python3 -m app.telegram.account_login
```

Интерактивный CLI:

- Имя аккаунта
- Прoxy (опционально)
- GPT prompt
- Телефон → SMS-код → 2FA
- session_string сохраняется в БД

11. Деплой

- Docker — docker compose up -d
- systemd — deploy/neurocomment.service + deploy/install-systemd.sh
- Логи — stdout + таблица logs в БД

12. Поток данных (итог)

Новый пост в канале

↓

post_listener (фильтр: канал, длина текста)

↓

comment_scheduler (лимит + % + сон)

↓

```
random delay 60-300 сек  
↓  
generator → GPT-4o-mini → комментарий  
↓  
comment_sender → linked-группа → Telegram  
↓  
comments (status: sent/failed) + logs
```

Каждый аккаунт проходит этот пайплайн независимо и параллельно, со своими настройками, prompt'ом и проху.